Lezione 9: Esercizi

Federica Montesano
federica.montesano@uniroma2.it



Indice della Sessione

1 Celebrity Problem

2 Rotazione Antioraria (Doubly Linked List)

3 Rilevazione e Rimozione di un Ciclo

4 Interleaving di una Coda (FIFO)

Roadmap

1 Celebrity Problem

2 Rotazione Antioraria (Doubly Linked List)

3 Rilevazione e Rimozione di un Ciclo

4 Interleaving di una Coda (FIFO)

Traccia dell'Esercizio

Consideriamo un evento a cui partecipano n persone, numerate da 0 a n-1. I rapporti di conoscenza reciproca tra i partecipanti sono modellati attraverso una matrice quadrata *mat* di dimensioni n x n, definita come segue:

- mat[i][j] == 1 indica che la persona i conosce la persona j.
- mat[i][j] == 0 indica che la persona i non conosce la persona j.

All'interno di questo gruppo, vogliamo determinare se è presente una "celebrità". Una persona è definita celebrità se e solo se soddisfa contemporaneamente le seguenti due condizioni:

1) Tutti gli altri partecipanti conoscono la celebrità.

2) La celebrità non conosce nessuno, tranne se stessa.

Si scriva una funzione in linguaggio C int findCelebrity(int** mat, int n) che riceva come argomenti la matrice **mat e la dimensione n, restituendo l'indice numerico della celebrità, qualora esista, altrimenti -1.

È obbligatorio implementare e utilizzare una struttura dati Stack basata su liste collegate dinamiche.

Rappresentazione dei Dati: Matrice delle Relazioni

L'elemento `mat[i][j]`

La relazione "i conosce j" è modellata da una matrice quadrata $N \times N$:

- mat$[i][j] == 1 \implies$ La persona i **conosce** la persona j.
- mat$[i][j] == 0 \implies$ La persona i **NON conosce** la persona j.

Interpretazione degli Assi nella Matrice

- **Riga** $i \rightarrow$ Azione compiuta da i (chi conosce).
- **Colonna** $j \rightarrow$ Azione subita da j (chi è conosciuto).

Celebrità $C \implies$ Riga C piena di 0 e Colonna C piena di 1 (eccetto diagonale)

Il Problema della Celebrità

Definizione Formale

In un gruppo di N persone $(0 \dots N-1)$, una persona C è definita **celebrità** se e solo se:

1. **Tutti la conoscono**: $\text{mat}[i][C] == 1, \quad \forall i \neq C$
2. **Non conosce nessuno**: $\text{mat}[C][i] == 0, \quad \forall i \neq C$

(Nota: Per convenzione $\text{mat}[i][i] == 1$ *per ogni persona).*

Il Problema della Celebrità

Definizione Formale

In un gruppo di N persone $(0 \dots N-1)$, una persona C è definita **celebrità** se e solo se:

1. **Tutti la conoscono**: $\text{mat}[i][C] == 1, \quad \forall i \neq C$
2. **Non conosce nessuno**: $\text{mat}[C][i] == 0, \quad \forall i \neq C$

(Nota: Per convenzione $\text{mat}[i][i] == 1$ per ogni persona).

Approccio Naive: $O(N^2)$

Verificare riga per riga e colonna per colonna ogni candidato nella matrice. Inefficiente per grandi gruppi!

Obiettivo Algoritmico: $O(N)$

Ridurre il problema a una scansione lineare in tempo $O(N)$ usando uno **Stack** per eliminare i candidati a coppie.

Fase 1 & 2: Il Torneo ad Eliminazione con lo Stack

Fase 1: Inizializzazione

Inserire tutti gli indici $0 \dots N-1$ nello Stack.
All'inizio ognuno è un potenziale candidato.

Fase 1 & 2: Il Torneo ad Eliminazione con lo Stack

Fase 1: Inizializzazione

Inserire tutti gli indici $0 \dots N-1$ nello Stack.
All'inizio ognuno è un potenziale candidato.

Fase 2: Torneo a Coppie

Finché lo Stack contiene ≥ 2 elementi, estrai indici A e B (`pop`):

- Se mat$[A][B] == 1 \implies A$ conosce B.
  $\rightarrow A$ **non** è celebrità! Scarta A, rimetti B (`push`).
- Se mat$[A][B] == 0 \implies A$ non conosce B.
  $\rightarrow B$ **non** è celebrità! Scarta B, rimetti A (`push`).

Fase 1 & 2: Il Torneo ad Eliminazione con lo Stack

Fase 1: Inizializzazione

Inserire tutti gli indici $0 \dots N-1$ nello Stack. All'inizio ognuno è un potenziale candidato.

Fase 2: Torneo a Coppie

Finché lo Stack contiene ≥ 2 elementi, estrai indici A e B (pop):

- Se $\text{mat}[A][B] == 1 \implies A$ conosce B.
  $\rightarrow A$ **non** è celebrità! Scarta A, rimetti B (push).
- Se $\text{mat}[A][B] == 0 \implies A$ non conosce B.
  $\rightarrow B$ **non** è celebrità! Scarta B, rimetti A (push).

Proprietà Fondamentale del Confronto

Ogni confronto $\text{mat}[A][B]$ elimina con certezza **esattamente un candidato**:

- Se A conosce B, A viola la condizione 2 (non deve conoscere nessuno).
- Se A non conosce B, B viola la condizione 1 (deve essere conosciuto da tutti).

Dopo $N-1$ passi rimane **un solo candidato**.

Fase 3: Validazione Finale

Attenzione!

Il vincitore del torneo della Fase 2 è l'**UNICO** potenziale candidato celebrità, ma **non è detto che sia davvero una celebrità**! (Potrebbe non esistere alcuna celebrità nel gruppo perché tutti si conoscono reciprocamente..).

Scansione di Validazione $O(N)$

Sia C il candidato estratto alla fine del torneo. Bisogna verificare per ogni $i \neq C$:

1. $\text{mat}[C][i] == 0$ (C non deve conoscere la persona i)
2. $\text{mat}[i][C] == 1$ (la persona i deve conoscere C)

Se anche una sola condizione fallisce $\implies$ **Restituisci** -1 (nessuna celebrità).
Se supera tutti i controlli $\implies$ **Restituisci** C.

Pseudocodice (Parte 1: Inizializzazione e Torneo)

Input: Matrice `mat[][]` di dimensione $n \times n$, Stack *stack*
Risultato *Indice della celebrità o* -1

```
// FASE 1: Inserimento iniziale nello stack
for i ← 0 to n − 1 do
    stack.push(i);
// FASE 2: Riduzione dei candidati a coppie
while size(stack) ≥ 2 do
    a ← stack.pop();
    b ← stack.pop();
    if mat[a][b] == 1 then
        stack.push(b) ;                          // 'a' conosce 'b' → 'a' scartato
    else
        stack.push(a) ;                          // 'a' non conosce 'b' → 'b' scartato
candidato ← stack.pop();
```

Pseudocodice (Parte 2: Validazione Finale)

Input: Intero $candidato$
Risultato *Restituisce* $candidato$ *se valido, altrimenti* -1

```
// FASE 3:  Validazione rigorosa del candidato
for i ← 0 to n − 1 do
    if i ≠ candidato then
        if mat[candidato][i] == 1 or mat[i][candidato] == 0 then
            return −1;
            ;                    // Candidato falso:  conosce qualcuno o non è conosciuto
return candidato;
;                                          // Superati tutti i controlli:  è la celebrità!
```

Analisi delle Complessità e Punti Critici

Complessità Algoritmica

- **Temporale**: $O(N)$
  - Fase 1 (Push iniziale): N operazioni $\rightarrow O(N)$
  - Fase 2 (Torneo): $N-1$ iterazioni $\rightarrow O(N)$
  - Fase 3 (Validazione): $2N$ controlli $\rightarrow O(N)$
- **Spaziale Ausiliaria**: $O(N)$ per contenere gli indici nello stack implementato come linked list.

Attenzione

Stiamo assumendo di avere le implementazioni di *push(elem e)* e $pop() \leftarrow e$.

Roadmap

1. Celebrity Problem
2. Rotazione Antioraria (Doubly Linked List)
3. Rilevazione e Rimozione di un Ciclo
4. Interleaving di una Coda (FIFO)

Traccia dell'Esercizio

Data una lista doppiamente collegata, scrivi una funzione in linguaggio C per ruotare la lista in senso antiorario di p nodi, dove p è un numero intero positivo dato ed è strettamente minore del numero totale di nodi presenti nella lista.
Input (Lista iniziale, con p = 2):
$NULL <--> A <--> B <--> C <--> D <--> E <--> NULL$
Output atteso (Dopo la rotazione antioraria di 2 posizioni):
$NULL <--> C <--> D <--> E <--> A <--> B <--> NULL$
Qual è la complessità temporale e spaziale nel caso peggiore (worst-case) della tua soluzione?

Strutturazione del Problema (esempio $p = 2$)

Prima di scrivere qualsiasi riga di codice, facciamo individuare visivamente alla classe i 3 nodi fondamentali:

1. **Nuova Coda (p-esimo nodo)**: Il nodo B (diventerà l'ultimo della nuova lista).
2. **Nuova Testa ($(p+1)$-esimo nodo)**: Il nodo C (diventerà il primo della nuova lista).
3. **Vecchia Coda**: L'ultimo nodo originario della lista, ovvero E.

Comprensione del Problema

Strutturazione del Problema (esempio $p = 2$)

Prima di scrivere qualsiasi riga di codice, facciamo individuare visivamente alla classe i 3 nodi fondamentali:

1. **Nuova Coda (p-esimo nodo)**: Il nodo B (diventerà l'ultimo della nuova lista).
2. **Nuova Testa ($(p+1)$-esimo nodo)**: Il nodo C (diventerà il primo della nuova lista).
3. **Vecchia Coda**: L'ultimo nodo originario della lista, ovvero E.

Focus

I primi p nodi (A e B) vengono staccati e traslati in fondo, agganciandosi dietro alla vecchia coda (E).

Pseudocodice dell'Algoritmo

Input: La testa della lista (`head`) e intero p
Risultato *La nuova testa della lista ruotata*

if *head* $==$ *NULL* **or** $p \leq 0$ **then**
  return head;
`// Individuazione del p-esimo nodo (nuova coda)`
$current \leftarrow head$;
$count \leftarrow 1$;
while $count < p$ **and** $current \neq$ *NULL* **do**
  $current \leftarrow current.next$;
  $count \leftarrow count + 1$;
if $current ==$ *NULL* **or** $current.next ==$ *NULL* **then**
  return head ; `// Se p è ≥ numero nodi, nessuna rotazione`
$newTail \leftarrow current$;
$newHead \leftarrow current.next$;

Pseudocodice dell'Algoritmo (continued)

```
// Troviamo la vecchia coda della lista
tail ← newHead;
while tail.next ≠ NULL do
    tail ← tail.next;
// Modifica dei Collegamenti
tail.next ← head ;                          // La vecchia coda punta alla vecchia testa
head.prev ← tail ;                  // Il prev della vecchia testa punta alla vecchia coda
newTail.next ← NULL ;                                  // La nuova coda chiude la lista
newHead.prev ← NULL ;                       // La nuova testa diventa l'inizio assoluto
return newHead;
```

Attenzione ai legami bidirezionali!

Nelle liste doppiamente collegate ogni spostamento richiede di spezzare e rifare **due legami** per ogni nodo. I passaggi chiave da rimarcare sono:

I passaggi algebrici dei puntatori

1. $tail \rightarrow next = head;$ $\rightarrow$ Collega la vecchia coda alla vecchia testa.
2. $head \rightarrow \text{prev} = tail;$ $\rightarrow$ Chiude il legame all'indietro simmetrico.
3. $newTail \rightarrow next = NULL;$ $\rightarrow$ Tronca la catena nel punto esatto della nuova coda.
4. $newHead \rightarrow prev = NULL;$ $\rightarrow$ Isola la nuova testa rimuovendo il legame con il blocco spostato.

Discussione sulla Complessità

Complessità

1. Temporale: O(p), con p numero di rotazioni, p è limitato dal numero totale di nodi N
2. Spaziale: O(1)

Roadmap

1 Celebrity Problem

2 Rotazione Antioraria (Doubly Linked List)

3 Rilevazione e Rimozione di un Ciclo

4 Interleaving di una Coda (FIFO)

Traccia dell'Esercizio

Data la testa (head) di una lista semplicemente collegata che potrebbe contenere un ciclo (ossia una situazione in cui l'ultimo nodo, anziché puntare a NULL, punta all'indietro verso un nodo precedente della stessa lista), scrivi una funzione in linguaggio C che:
Verifichi se nella lista è presente un ciclo. Se il ciclo esiste, lo rimuova interrompendo il legame circolare (ripristinando il puntatore next dell'ultimo nodo a NULL), in modo da restituire una lista lineare standard valida.
Se il ciclo non è presente, lasci la lista inalterata.
Qual è la complessità temporale e spaziale nel caso peggiore (worst-case) della tua soluzione?

Comprensione del Problema

Il Pericolo del Loop Infinito

Un normale attraversamento lineare su una lista con un ciclo non terminerà mai (NULL non verrà mai raggiunto), mandando in stallo il programma.

Comprensione del Problema

Il Pericolo del Loop Infinito

Un normale attraversamento lineare su una lista con un ciclo non terminerà mai (NULL non verrà mai raggiunto), mandando in stallo il programma.

Il Problema

Per risolvere questo esercizio bisogna:

1. **Verificare** se il ciclo esiste.
2. Trovare il nodo iniziale del ciclo
3. **Rimuoverlo** spezzando il legame circolare (next = NULL).

Fase 1: Rilevazione del Ciclo (Tartaruga e Lepre)

I Puntatori Slow & Fast

Si utilizzano due puntatori che partono entrambi dalla `head`:

- **Slow (Tartaruga)**: Avanza di **1 passo** (slow = slow $\rightarrow$ next).
- **Fast (Lepre)**: Avanza di **2 passi** (fast = fast $\rightarrow$ next $\rightarrow$ next).

Rilevazione del ciclo

- Se la lista è lineare, `fast` arriverà a `NULL` ($\rightarrow$ nessun ciclo).
- Se è presente un ciclo, `fast` "doppierà" `slow` facendo un giro e i due puntatori **si incontreranno** (slow == fast).

Fase 2: Trovare l'Inizio del Ciclo e Rimozione

Trovare il primo nodo del ciclo

Una volta che $slow == fast$:

1. Si riporta `slow` **alla** `head` della lista.
2. Si lasciano muovere **sia** `slow` **che** `fast` **alla stessa velocità** (1 passo alla volta).
3. **Il punto in cui si incontrano di nuovo** è esattamente l'**inizio del ciclo**[a]!

[a] Questo in realtà non è banale e richiederebbe una dimostrazione.. Se vi interessa si chiama Algoritmo di Floyd

Rimozione del Ciclo

Trovato l'inizio del ciclo, basta scorrere con un puntatore all'interno del loop fino a individuare l'ultimo nodo (il cui `next` punta all'inizio) e impostarlo a `NULL`.

Pseudocodice (Parte 1: Rilevazione Ciclo)

Input: La testa della lista (`head`)
Risultato *Rileva la presenza di un ciclo*

$slow \leftarrow head$;
$fast \leftarrow head$;
`// FASE 1: Ricerca dello scontro (Tartaruga e Lepre)`
while $fast \neq NULL$ **and** $fast.next \neq NULL$ **do**
    $slow \leftarrow slow.next$;
    $fast \leftarrow fast.next.next$;
    if $slow == fast$ **then**
        break ; `// Ciclo rilevato!`
if $fast == NULL$ **or** $fast \rightarrow next == NULL$ **then**
    return ; `// Lista lineare, nessun ciclo presente`

Pseudocodice (Parte 2: Trovare l'Inizio e Rimuoverlo)

Input: Puntatore `fast` e puntatore `head`
Risultato *Lista lineare senza cicli*

```
// FASE 2: Individuazione inizio del ciclo e rimozione
slow ← head;
if slow == fast then
    ;                                          // Caso speciale: ciclo direttamente sulla head
    while fast.next ≠ slow do
        fast ← fast.next;
    fast.next ← NULL;
    return;
// Caso generale: entrambi i puntatori avanzano un passo alla volta
while slow.next ≠ fast.next do
    slow ← slow.next;
    fast ← fast.next;
fast → next ← NULL ;                                   // Spezziamo il ciclo sulla coda
```

Discussione sulla Complessità

Complessità

Complessità Temporale: O(N), facciamo due scansioni lineari (O(2N))
Complessità Spaziale: O(1)

Attenzione

Essendo questo esercizio basato su un algoritmo noto e molto difficile da risolvere autonomamente, se fosse un esercizio di esame verrebbe fornito lo pseudocodice

Roadmap

1. Celebrity Problem
2. Rotazione Antioraria (Doubly Linked List)
3. Rilevazione e Rimozione di un Ciclo
4. Interleaving di una Coda (FIFO)

Traccia dell'Esercizio

Data una struttura dati Coda (Queue) di dimensione n pari, implementata tramite liste collegate dinamiche, si scriva una funzione in linguaggio C che riorganizzi la coda intervallando (interleaving) la sua prima metà con la seconda metà.
L'interleaving consiste nel posizionare alternativamente un elemento proveniente dalla prima metà della coda e un elemento proveniente dalla seconda metà, mantenendo inalterato l'ordine relativo originario all'interno di ciascuna metà. La soluzione deve essere sviluppata rispettando l'incapsulamento della struttura Coda (utilizzando le operazioni primitive di enqueue() e dequeue()).
Esempi:
- Input: q = [2, 4, 3, 1] (Dimensione n = 4, prima metà: [2, 4], seconda metà: [3, 1])
- Output: [2, 3, 4, 1]
- Input: q = [3, 5] (Dimensione n = 2, prima metà: [3], seconda metà: [5])
- Output: [3, 5]

Interleaving di una Coda: Definizione

Il Problema

Data una coda (Queue) di dimensione N pari, implementata tramite liste collegate dinamiche, vogliamo riorganizzarla **intervallando (interleaving) la sua prima metà con la seconda metà**.

Esempio Visivo

Ingresso: $[1, 2, 3, 4, 5, 6]$

Uscita: $[1, 4, 2, 5, 3, 6]$ *(Alternanza simmetrica tra prima e seconda metà)*

Vincolo Architetturale Fondamentale

La soluzione deve rispettare rigorosamente l'**incapsulamento della struttura Coda** FIFO, utilizzando esclusivamente le operazioni primitive di `enqueue()` e `dequeue()`.

Strategia di Risoluzione tramite Code Ausiliarie

Passaggi dell'Algoritmo

1. **Calcolo della dimensione**: Si calcola la dimensione N. Se N è dispari o zero, l'interleaving simmetrico non è possibile.
2. **Creazione di code ausiliarie**: Si creano due code ausiliarie (`firstHalf` e `secondHalf`).
3. **Estrazione Prima Metà**: Si prelevano i primi $N/2$ elementi dalla coda principale e si accodano in `firstHalf`.
4. **Estrazione Seconda Metà**: Si prelevano i successivi $N/2$ elementi e si accodano in `secondHalf`.
5. **Fusione (Interleaving)**: Si preleva un elemento alternato da ciascuna coda ausiliare e si rimette in `q`.

Pseudocodice dell'Interleaving

Input: La coda principale q
Risultato *Coda q riorganizzata a coppie alternate*

Calcola la dimensione N di q;
if $N \pmod 2 \neq 0$ **or** $N == 0$ **then**
  return ; `// Richiesta dimensione pari`
$halfSize \leftarrow N/2$;
Inizializza due code ausiliarie: `firstHalf`, `secondHalf`;
for $i \leftarrow 0$ **to** $halfSize - 1$ **do**
  firstHalf.enqueue(q.dequeue());
for $i \leftarrow 0$ **to** $halfSize - 1$ **do**
  secondHalf.enqueue(q.dequeue());
while *!isEmpty*(*firstHalf*) **and** *!isEmpty*(*secondHalf*) **do**
  q.enqueue(firstHalf.dequeue());
  q.enqueue(secondHalf.dequeue());

Analisi delle Complessità e Punti Critici

Complessità Algoritmica

- **Temporale**: $O(N) \rightarrow$ Scansione iniziale per il conteggio O(N), smistamento nelle code O(N) e fusione finale O(N) lineari.
- **Spaziale Ausiliaria**: $O(N) \rightarrow$ Le due code ausiliarie contengono complessivamente tutti gli N elementi.

Attenzione

- **Rispetto dell'Incapsulamento**: Non accedere mai ai puntatori interni `front`/`rear` delle code di supporto; usare solo `enqueue` e `dequeue`.
- **Gestione Dimensione Dispari**: Se N è dispari, come cambia la ripartizione degli elementi per garantire interleaving finale (non simmetrico)?